

AykenOS Geliştirme Yol Haritası

AykenOS; yapay zekâ destekli, çoklu mimariyi hedefleyen yenilikçi bir işletim sistemi projesidir. Bu proje klasik işletim sistemi kavramlarını, veri-odaklı ve AI bütünleşik bir yaklaşımla yeniden ele almaktadır. Klasik `mkdir`, `cd`, `ls` komutlarıyla sembolize edilen geleneksel kabuk anlayışı yerine, “**AI-native shell + veri işletim sistemi**” diyebileceğimiz bütünleşik bir yapı hedeflenmektedir – bu fikir oldukça güçlü ve sıradışıdır. AykenOS, modüler bir çekirdek üzerine inşa edilmiş olup, çekirdek seviyesinde ve kullanıcı alanında **küçük yapay zekâ modellerinin (TinyLLM) entegrasyonunu** içerir. Dosya sistemi, **klasörlerden ziyade anlamlı veri konteynerleri** barındıran hibrit bir modelle tasarlanacak; böylece POSIX uyumluluğu korunurken, kabuk dilinde dosyalar **tiplenmiş, AI-farkında veri varlıkları** olarak görünecektir. Aşağıdaki yol haritası, bu vizyonu adım adım gerçekleştirmek için teknik ekibin izlemesi gereken aşamaları, doğru öncelik sırasıyla sunmaktadır. Plan, her adımda ekibin öğrenmesi gereken teknolojilere de değinmekte ve proje tanıtımı sırasında kavramların net anlaşılması için gerekli açıklamaları içermektedir. Her faz, bağımlılıkları en aza indirerek basitten karmaşığa doğru yapılandırılmış ve test edilebilir parçalar halinde planlanmıştır.

Faz 1: Çekirdek Temelinin Tamamlanması ve Stabilizasyonu

İlk adım, AykenOS çekirdek altyapısını **kararlı bir temel** haline getirmektir. Halihazırda UEFI önyükleme ile çekirdek yükleme, temel bellek yönetimi ve basit bir framebuffer konsolu başarıyla çalışmaktadır. Ancak kullanıcı moduna geçiş ve bazı çekirdek bileşenleri henüz tamamlanmamıştır. Bu fazda ekibin öncelikleri şunlar olmalıdır:

- **Bellek ve Önyükleme Haritalaması:** UEFI bootloader içinde yüksek adres alanı (higher-half) çekirdek çalıştırılması için gereken sayfa tablosu (PML4) oluşturulmalı, `kernel.elf` yüklenirken bu haritalar hazırlanmalıdır. Özellikle `boot_info.pml4_phys` alanının doğru doldurulması ve çekirdek başlangıcında identity mapping’in kaldırılması sağlanmalıdır. Bu adım, kullanıcı modunu başlatabilmek için kritik önemdedir.
- **Sistem Çağrıları ve Kullanıcı Modu Geçişi:** Çekirdekte bir **sistem çağrısı (syscall) mekanizması** kurulmalıdır. Mevcut durumda `INT 0x80` kesmesi `ENOSYS` dönmektedir, yani hiçbir sistem çağrısı tanımlı değil. En azından temel çağrılar (`read`, `write`, `open`, `close`, `exit` vb.) dispatch edilerek kullanıcı programlarının kernel servislerine erişimi sağlanmalıdır. Ardından, **kullanıcı moduna geçiş** için gerekli altyapı tamamlanmalıdır: Yeni bir kullanıcı PML4 oluşturup Ring3 modunda basit bir süreç başlatma (örneğin PID1 olarak bir `init` süreci) gerçekleştirilmelidir. Bu, AykenOS'un ilk kez kendi üzerinde çalışan bir kullanıcı alanı programı yürütmesini gösterecek önemli bir adımdır.
- **Zamanlayıcı ve Çoklu Görev:** Mevcut çekirdek prototipinde kooperatif bir scheduler iskeleti bulunmakta, ancak **preemptive multitasking** henüz tam uygulanmamıştır. Bu aşamada PIT (programmable timer) kesmeleri ile scheduler entegre edilmeli, `sched_add_task` işlevi gerçek süreçleri kuyruğa alacak şekilde doldurulmalıdır. Başlangıçta round-robin ya da cooperative yield stratejilerinden biri seçilerek en azından birden fazla görev arasında geçiş testi yapılabilir. Örneğin, PID1 `init` süreci

yanında basit bir AI servis stub süreci (PID2) yaratılarak context switching başarılı şekilde doğrulanmalıdır.

- **Aygıt ve Sürücü Temelleri:** Çekirdek, şu an temel bir framebuffer konsolu ve programlanabilir interrupt denetleyicisi (PIC) ile timer sürücüsünü içeriyor. Bunu genişletmek için **Device FileSystem (DevFS)** altyapısı kurulmalıdır. Örneğin, klavye girdisi için basit bir sürücü ve `/dev/keyboard` benzeri bir arayüz, seri port için `/dev/serial` gibi arabirimler eklenebilir. Şu anda DevFS iskeleti boş durumda olduğundan, çekirdek başlangıcında dummy bir cihaz düğümü ekleyecek bile olsa bir **cihaz kayıt API**'i oluşturulmalıdır. Bu sayede ileride gerçek donanım sürücüleri eklendiğinde alt yapı hazır olacaktır.
- **Test ve Doğrulama:** Faz 1'in çıktısı, QEMU ortamında (ve mümkünse gerçek donanımda) **kararlı şekilde önyüklenabilen ve çok temel kullanıcı programını çalıştırabilen** bir OS olmalıdır. Ekip, her adımda test odaklı ilerlemelidir: Örneğin, bootloader çıktılarını ve bellek haritalarını kontrol etmek (PML4 fiziksel adresi doğru mu, identity map kaldırıldı mı), basit bir `init` sürecinin `printf` ile mesaj basabildiğini gözlemek önemlidir. Zamanlayıcı kesmesiyle context switch'in sorunsuz gerçekleştiği, ya da en azından kernel panik olmadan birden fazla görevi idare edebildiği doğrulanmalıdır. Bu aşamadaki testler, sorunları erken yakalayarak sonraki fazlarda daha karmaşık özelliklere geçmeden önce sağlam bir temel kurulmasını sağlayacaktır.

Bu ilk faz, ekibin **işletim sistemi geliştirme temellerini** pekiştireceği dönemdir. C ve assembly dilinde uzmanlaşma, mimari özelliklerin (x86_64 paging, kesme tabloları vb.) anlaşılması bu süreçte kazanılacaktır. Rust diline bu aşamada çekirdek içinde zorunlu ihtiyaç yoktur (çekirdek büyük oranda C ile geliştirilmiştir), ancak ileride Rust bileşenleri entegre edileceği için ekibin Rust ortamını (örn. Cargo, `no_std` programlama) kurması ve denemeler yapması tavsiye edilir.

Faz 2: Veri-Odaklı Dosya Sistemi ve Yeni Kabuk Mimarisi (AI'sız Temel)

Çekirdek stabil hale geldikten sonra, AykenOS'un asıl yenilikçi yönlerini inşa etmeye başlayabiliriz. Bu fazda amaç, **geleneksel dosya sistemi kavramını zenginleştirerek veri merkezli bir katman** oluşturmak ve buna uygun bir **kabuk (shell) dilinin temelini** atmaktır – başlangıçta yapay zekâ entegrasyonu olmadan, yani kuralları biz tanımlayarak.

- **Tiplenmiş Veri Nesneleri ve Meta-Filesystem:** Klasik dosya sistemi dizinleri, AykenOS'ta "**veri konteyneri**" haline dönüşecek. Bu, her dizinin bir türü ve şeması olacağı anlamına gelir. Örneğin `users:tabular` adlı bir veri kümesi oluşturulduğunda, onun bir şeması (sütun adları ve tipleri), erişim izinleri, indeksleme bilgileri ve AI ile ilgili özet/embedding alanı meta-veri olarak tanımlanacak. Bu bilgileri depolamak için bir **meta-veri deposu** tasarlamalıyız. JSON formatında bir meta veritabanı veya her dizine eşlik eden bir meta dosya kullanılabilir. Nitekim proje tartışmalarında verilen örnek JSON, bir tablo türü veri nesnesinin kimlik, tür, kategori, şema, depolama formatı ve AI parametrelerini içermektedir. Bu yapıyı AykenOS içinde uygulamak, belki basit bir **yerleşik veritabanı** (mini bir JSON/XML parser veya anahtar-değer deposu) geliştirmeyi gerektirecektir.
- **POSIX Uyumunu Koruma:** Veri odaklı bu yaklaşım, altta yatan dosya sistemini tamamen ortadan kaldırmaz, aksine onu zenginleştirir. Hedef, **hibrit bir FS** oluşturmaktır: Harici programlar ve POSIX araçları AykenOS'un dosya sistemini

klasik anlamda (düz dosyalar ve dizinler) görmeye devam edebilmeli, ancak AykenOS kabuğunda kullanıcıya **anlamli veri varlıkları** sunulacaktır. Örneğin, raporlar adlı bir dizin POSIX üzerinden erişildiğinde sıradan bir klasör gibi davranırken, AykenOS kabuğu içinde bu dizin belki de `raporlar:tabular` tipinde bir veri tablosu olarak özel komutlarla yönetilebilecektir. Bunu sağlamak için, meta-veri katmanı ile klasik FS arasında senkronizasyon olmasına dikkat edilmelidir (örn. bir dizin silindiğinde meta kaydı da silinmeli vs.).

- **Yeni Kabuk (Shell) Söz Dizimi ve Yorumlayıcı:** Bu aşamada belki projenin en öğretici kısmı, **yenilikçi bir komut satırı dilinin tasarımı ve gerçekleştirilmesidir**. Klasik shell (bash, etc.) yerine, iç içe geçmiş hiyerarşik komutları destekleyen bir yorumlayıcı geliştirilecek. Tasarlanan söz diziminde `>` işareti global bağlamı, `>>` alt bağlamı (belirli bir veri konteyneri veya işlemi içine girildiğini) temsil edecek; `>[ ]` ise paralel dallanmalar ya da pipeline açılımlarını gösterebilecektir. Örneğin:
 - `> data.users` komutuyla **users** veri koleksiyonunu bağlam olarak seçip,
 - `>> add {...}` ile bu koleksiyona bir kayıt eklemek,
 - veya `>> query 'role=="admin"'` ile filtreleme yapmak mümkün olacaktır.

Bu yeni shell konseptini uygulamak için öncelikle **basit bir parser ve yorumlayıcı** yazılması gerekir. Faz 2'nin hedefi, **henüz AI entegrasyonu olmadan**, kabuğun temel komutlarını çalıştıran bir iskeletini oluşturmaktır. Öneri olarak, ilk etapta 1-2 veri türüyle sınırlı tutmak akıllıcadır (örneğin sadece `tabular` ve basit bir `log/metin` türü). Böylece karmaşıklık kontrol altında tutulur. Yorumlayıcı, kullanıcının girdiği komutları parse edip ilgili kernel veya kullanıcı alanı fonksiyonlarına çağrı yapacaktır. Bu kabuk muhtemelen kullanıcı modunda çalışan bir uygulama (örneğin `ayken-shell`) olarak implemente edilecektir; çekirdek ile etkileşim için sistem çağrılarını kullanabilir veya özel bir shell daemon'ı ile haberleşebilir.

- **Tasarımın Belirlenmesi (Spesifikasyon):** Bu faza başlamadan önce, ekibin küçük bir **tasarım belgesi** hazırlaması önerilir. Kabuk dili söz diziminin BNF benzeri bir tanımı, desteklenecek ilk komut seti (ör. `data.create`, `data.add`, `data.query` gibi) ve veri modeli (hangi tip veri nesneleri, meta veri alanları vs.) netleştirilmeli. Bu sayede tüm ekip aynı anlayışla geliştirmeye başlayabilir. Örneğin, Faz 1 - Çekirdek model (LLM'siz) başlığı altında bir prototip kabuk ve veri modeli tarif edilmiştir. Bu tarifler, gerçek koda geçmeden evvel beklentileri belirler:
 - `data.create users:tabular {...}` gibi komutlar için gereken söz dizimi ve işlev,
 - `data.add`, `data.query` gibi işlemlerin parametre yapısı,
 - Parser'in destekleyeceği hiyerarşik prompt şekilleri (`>`, `>>`, `>[ ]`) vb..
- **Uygulama ve Test:** Tanımlanan bu altyapıyı uygularken ekip muhtemelen C dilini kullanarak bir kabuk yorumlayıcısı geliştirebilir. Alternatif olarak, Rust dilinde bir kullanıcı alanı uygulaması yazıp bunu çekirdeğe minimal bir C arabirimiyle bağlamak da mümkündür. Hangi dil kullanılırsa kullanılsın, amaç **modüler ve genişleyebilir** bir yapı kurmaktır. Örneğin, her veri türü için işlemler ayrı modüller olarak yazılabilir (`tabular` modülü, `text` modülü vb.). Test aşamasında, gerçek bir dosya sistemi entegre edilmemiş olsa bile RAM içinde sanal bir veri deposu kullanılarak komutların doğru çalıştığı doğrulanmalıdır. Örneğin:
 - `data.create users:tabular {schema:{...}}` komutu sanal meta-veritabanında bir kayıt oluşturuyor mu?
 - Ardından `data.users.add {...}` denendiğinde, veri yapısına eklendiği görülebiliyor mu?

- o `data.users.query "role=='admin'"` basit bir filtre uygulayıp doğru sonuç döndürüyor mu?

Bu testleri konsolda çıktılarla doğrulamak önemlidir. Henüz AI entegre edilmediğinden, beklenen çıktıları manuel hesaplayıp kabuğun ürettikleriyle karşılaştırabiliriz.

Bu faz, ekibin **dosya sistemi tasarımı, veri yapıları ve dil işleme** konularında bilgisini derinleştirecektir. Ayrıca, modüler programlama pratiği kazanılacak, çünkü her yeni veri türü veya komut eklemesi, kabuğa eklenecek bir *plugin* gibi düşünülmelidir. Faz 2 tamamlandığında AykenOS, geleneksel bir OS çekirdeği üzerinde çalışan fakat kendine özgü bir kabuk ve veri katmanına sahip prototip bir işletim sistemi olacaktır. Bu, dışarıya proje vizyonunu anlatmak için somut bir demo da sunabilir: Küçük bir sanal terminalde geleneksel olmayan komutlarla veri oluşturup sorgulayan, POSIX'ten farklılıklar gösteren bir OS kabuğu.

Faz 3: Yapay Zekâ Entegrasyonu (TinyLLM) ile Akıllı Kabuk ve Veri İşleme

AykenOS'un en ayırt edici yanı, bünyesindeki yapay zekâ ajanları olacaktır. Faz 3'te, Faz 2'de kurulmuş olan veri-kabuk altyapısını artık **küçük dil modelleriyle entegre ederek** akıllı hale getireceğiz. Bu entegrasyon kademeli ve kontrollü bir şekilde yapılmalıdır:

- **AykenCore AI Modüllerinin Hazırlanması:** Projenin Rust tabanlı AI çekirdeği "ayken-core" dizininde halihazırda tanımlanmış olan **ABDF (Ayken Binary Data Format)** ve **BCIB (Binary Compressed Instruction Bundle)** gibi formatlar, AI modellerinin derlenip çalıştırılmasıyla ilgilidir. Bu aşamada ekip, küçük bir dil modelini (örneğin parametre boyutu küçük, yerel bir modeli) AykenOS ortamında çalıştırmaya hazır hale getirmelidir. Gerekirse model, ABDF formatına dönüştürülüp initrd veya disk imajına eklenecek ve OS içinde *tinyLLM runtime* tarafından yüklenip çalıştırılacaktır. AykenOS çekirdeğinde **AykenCoreLM** adıyla bir dil modeli arayüzü ve tokenizer yapıları zaten öngörülmüş durumdadır. İlk hedef, bu modeli kullanıcı modunda bir servis olarak çalıştırmak veya çekirdeğe gömülü basit bir çağrı mekanizmasıyla kullanıma sunmaktır.
- **Doğal Dil Sorgular için Aracılık:** Kabuk diline, AI desteğini devreye sokacak bir söz dizimi eklemeyi planlıyoruz. Önerilen yöntem, kullanıcının bir komuttan önce ? veya benzeri bir operatör kullanarak AI yardımını talep etmesidir. Örneğin kullanıcı `> ? "projeleri listele"` yazdığında, shell bu isteği bir **Shell LLM** modülüne iletecek. Shell LLM'in görevi, kullanıcının doğal dildeki niyetini anlayıp bunu kabuğun anlayacağı komutlara dönüştürmektir. Ancak kritik nokta şudur: **Son kararı veren komut yorumlayıcı olmalıdır**; LLM sadece bir öneri sunar. Bu ilke, sistem güvenliği için önemlidir (LLM çıktısını asla doğrudan çalıştırmıyoruz). Faz 3'te öncelikle bu mekanizmayı basit bir örnekle gerçekleştirebiliriz:
 - o Örneğin, `> data.users.ai.ask "admin kullanıcıları özetle"` komutunu ele alalım. Shell, `'admin kullanıcıları özetle'` şeklindeki doğal dil talebini AI modülüne iletecek.
 - o TinyLLM modeli bu isteği analiz edip, ilgili veri kümesindeki admin rolüne sahip kullanıcıları özetlemek için bir plan üretecek. Bu plan, mesela `"SELECT`

* FROM users WHERE role='admin'" benzeri bir sorgu ya da kabukta art arda çalışacak komut dizisi olabilir.

- Shell, LLM'den gelen bu öneriyi alıp bir komut gibi çalıştıracak ve sonucu kullanıcıya sunacaktır. Özetle, LLM doğal dili alıp kabuğun anlayacağı dile çeviren bir **ara katman** olacak.
- **Veri ve Donanım İçin Uzman AI Modelleri:** AykenOS içinde farklı görevler için özelleşmiş küçük AI ajanları konsepti bulunmaktadır. Bu fazda Shell LLM dışında en az bir modeli daha entegre etmeyi planlayabiliriz: **Donanım monitörü AI (HW Agent)**. Donanım ajanı, CPU, bellek, disk, sıcaklık gibi telemetri verilerini toplayıp bunları özetleyebilen veya doğal dil ile açıklayabilen bir modeldir. Örneğin kullanıcı > sys.hw.status dediğinde anlık önemli metrikleri görür; > sys.hw.ai.explain "Neden yavaş?" dediğinde ise bu telemetri verilerini yorumlayan bir yanıt alır. Bu yanıt, örneğin "CPU kullanımınız yüksek ve bellek sıkışmış görünüyor, bu yüzden sistem yavaşlıyor olabilir" şeklinde doğal dil bir açıklama olabilir. Yine burada da AI'nın önerileri sadece bilgi amaçlı olacak, **otomatik aksiyon almayacak** (örneğin kendisi gidip CPU frekansını değiştirmeyecek). Donanım ajanının çıktıları, kullanıcının onayı veya önceden tanımlı politika olmadan sisteme uygulanmamalıdır.
- **AI Entegrasyonunun Mimarisi:** Bu aşamada önemli bir karar, AI modellerinin **nerede ve nasıl çalışacağı** konusudur. Güvenlik ve kararlılık açısından, TinyLLM motorlarını kullanıcı modunda izole etmek iyi bir tercih olacaktır. Önerilen mimaride, çekirdek boot ettikten sonra PID1 olarak `ayken_init` süreci başlıyor ve ardından birtakım servisleri yüklüyor olabilir. Örneğin:
 - `ayken-fs` servisi: Veri katmanını yöneten süreç (veya kernel modülü).
 - `ayken-ai-core` servisi: AI modelinin runtime'ını barındıran (model yükleme, inferencing yapan) süreç.
 - `ayken-hw-agent` servisi: Donanım telemetrisi toplayan ve AI açıklama üreten süreç.
 - `ayken-shelld` servisi: Bir kabuk sunucusu/daemoni, çoklu kabuk istemi olursa yönetir.
 - Son olarak kullanıcı interaktif bir terminal açtığında `ayken-shell` programı çalışır, bu program gerekirse `ayken-shelld` ve `ayken-ai-core` ile iletişime geçerek komutları işleyecektir.

Bu dağıtık yapı sayesinde, **AI olmadığı durumda da** sistemin çalışabilmesi mümkün olur. Nitekim tasarım gereği AykenOS kabuğu, AI modelleri yüklenmemiş olsa dahi sadece kural tabanlı olarak (Phase 2'deki gibi) çalışabilmelidir. AI servisleri mevcut olduğunda ise kabuk, onlardan ekstra "beyin" desteği alacaktır. Bu mimari, güvenlik için de faydalıdır; zira AI işlemleri ayrı süreçte izole edilerek, olası bellek hatalarının çekirdeği etkilemesi önlenir.

- **Eğitim ve Performans Konuları:** Ekip bu aşamada yapay zekâ entegrasyonu ile ilgili pek çok yeni teknolojiye adım atacaktır. **Doğal dil işleme, küçük dil modellerinin eğitimi** veya uyarlanması, **model sıkıştırma/optimizasyonu** (gerekirse TinyLLM'nin CPU üzerinde makul hızda çalışması için) gibi konular gündeme gelecek. AykenOS kendi modelini geliştirmek zorunda değil – mevcut açık kaynak 50-100M parametrelili bir modeli alıp uyarlayabilir. Ancak ABDF/BCIB gibi formatlar özel bir derleyici veya dönüştürücü gerektiriyorsa, ekip bunun araçlarını Rust ile zaten geliştirmiş durumda. Bu araçlar kullanılarak model dosyası hazırlanabilir. Performans için, **inference döngüsünde optimizasyon** kritik olacak: Docx içindeki önerilerde, LLM kullanımında prompt boyutunu minimal tutmak ve sonuçları cache'lemek gibi

teknikler belirtilmiş. Ekip, her AI özelliğini eklediğinde sistemin belirgin yavaşlamalar yaşamadığını, özellikle de etkileşimli kullanımda makul gecikmeler olduğunu test etmelidir. Gerekirse C++ ile derlenmiş optimize kütüphaneler veya SIMD hızlandırmalar kullanılabilir (Rust içinde de SIMD desteği veya Crate'ler ile bu sağlanabilir).

- **Test ve Demo:** Bu faz tamamlandığında AykenOS'un dışı karşı en cazip yönü ortaya çıkmış olacak. Testler sadece teknik doğrulama için değil, projenin sunumu için de düşünülmelidir. Örneğin, bir demo senaryosu:

1. `> data.users.query 'role == "student"'` komutu ile klasik sorgu,
2. `> data.users.ai.ask "Öğrencilerin not ortalamalarını özetle"` komutu ile AI destekli sorgu,
3. `> sys.hw.status` ve ardından `> sys.hw.ai.explain "neden yavaş"` komutları ile sistem durumunun AI tarafından açıklanması,
4. Hatta `> ? "diskte en çok yer kaplayan dosyalar hangileri?"` gibi genel bir soruya kabuğun yanıt üretmesi (LLM aracılığıyla bir dizi `find/sort` komutu çalıştırarak).

Bu tip akıllı işletim sistemi etkileşimlerini göstermek, AykenOS'un **AI-native OS** vizyonunu somutlaştıracaktır. Test sırasında güvenlik açısından da kontrol yapılmalı: AI önerileri mantıksız veya potansiyel tehlikeli bir komut üretiyorsa, kabuk bunu çalıştırmadan önce ya engellemeli ya da kullanıcı onayı istemelidir (bu da ayrı bir test konusu: örneğin AI bir dosya silmeyi önerirse, kabuk "LLM komutu `rm` önermek üzere, onaylıyor musunuz?" gibi bir mekanizma ile araya girmeli).

Bu faz sonunda ekip, **sistem programlama ile yapay zekâyı birleştirme** konusunda özgün bir deneyim kazanmış olacak. Proje, sadece bir işletim sistemi değil, aynı zamanda sınırlı kaynaklarla çalışan entegre bir AI platformu haline gelecektir. Bu hem araştırma açısından çok değerli (literatürde de yeni yeni tartışılan *AI-native OS* konseptine uygun) hem de potansiyel kullanıcılar için heyecan verici bir üründür medium.com. AykenOS, AI'yı bir eklenti olarak değil, tasarımın merkezinde bir bileşen olarak ele alarak çağdaş işletim sistemlerinden ayrılmaktadır medium.com.

Faz 4: Donanım Entegrasyonu ve Görsel Dashboard Arayüzü

Bu aşamada, AykenOS'un hem düşük seviyede donanım desteğini genişletip hem de yüksek seviyede kullanıcı arayüzüne ilk adımı atacağız. Hedef, **sistemin farklı platformlarda çalışabilirliğini artırmak** ve kabuk deneyimini zenginleştirmek için **grafiksel geri bildirim imkânı** sunmaktır.

- **Genişletilmiş Donanım Desteği (Sürücüler):** AykenOS çekirdeği başlangıçta PC ortamı (x86_64 UEFI) için geliştirilmiştir. Projede ARM64, RISC-V ve hatta mikrodenetleyici bootloader kodları bulunuyor, ancak çekirdek seviyesinde bu mimarilere tam destek sağlanması gerekecek. Bu fazda, en azından **Raspberry Pi (ARM tabanlı)** gibi bir platformda AykenOS'u çalıştırmayı amaçlıyoruz. Bunun için:
 - Mevcut ARM64 bootloader'ı kullanarak Pi üzerinde kernel önyükleme denemeleri yapılmalı.
 - ARM mimarisine özgü kesme denetimi, bellek haritalaması vb. konularda kernel kodunda gerekli şart blokları veya ayrı derlemeler eklenmeli (Örn:

- x86_64 için yazılmış assembly rutinlerinin ARM karşılıkları geliştirilmeli, halihazırda `arch/arm64` dizini veya benzeri bir yapı oluşturulabilir).
- RPi özelinde, video çıkışı (framebuffer) ve giriş (USB klavye) gibi temel aygıtlar için sürücüler gerekebilir. Başlangıç olarak RPi'nin basit framebuffer'ını ve USB HID sürücüsünü (veya seri konsol üzerinden kontrol) sağlayarak, en azından kabuk orada da çalışır hale getirilebilir.
 - **Mobil Sistemlere Entegrasyon:** RPi ile ilk adım atıldıktan sonra, AykenOS'u gerçek bir mobil cihazda çalıştırmak uzun vadeli bir hedef olabilir. Mobil entegrasyon deyince, iki yönlü düşünmek lazım:
 1. **AykenOS'u Mobil Donanımda Çalıştırma:** Örneğin bir ARM tabanlı akıllı telefona veya tablete AykenOS'u port etmek. Bu oldukça zorlu bir görev, zira kapalı donanım sürücüleri ve grafik ara yüz gibi konular var. Ancak, açık geliştirme kartları (PinePhone, eski Android telefonlar vb.) üzerinde denenebilir. Bu kapsamda dokunmatik ekran sürücüleri, güç yönetimi (pil durumu), kablosuz haberleşme gibi bileşenler çok ciddi çalışma gerektireceği için, planın bu aşamasında tam mobil desteği hedeflemek riskli olur. Bunun yerine, altyapının **modüler** tasarlanmasıyla mobil desteğin önü açılır. Yani, platform bağımlı kısımlar soyutlanır, böylece bir mobil port yapmak isteyen ileride sadece o katmanı yazabilir.
 2. **Mobil Cihazlarla Etkileşim/Entegrasyon:** Belki daha kısa vadede, AykenOS'un üretkenliği için mevcut mobil cihazları kullanmak söz konusu olabilir. Örneğin, AykenOS üzerinde çalışan bir hizmetin bir akıllı telefon uygulamasıyla iletişim kurması, ya da telefonu sensör veri kaynağı olarak kullanmak gibi. Bu tür senaryolar için AykenOS'a ağ veya Bluetooth gibi iletişim yetenekleri kazandırmak gerekir (aşağıda ağ desteği konusunda değinilecektir). Dolayısıyla Faz 4'te kesin olmasa bile, bir **ağ yığını (TCP/IP)** entegrasyonunu yavaştan düşünebiliriz, çünkü mobil entegrasyon genelde ağ üzerinden haberleşmeyi içerir.
 - **Gerçek Zamanlı Telemetri ve Dashboard:** Faz 3'te komut satırından donanım durumunu sorgulama ve AI ile yorumlama yapmıştık. Şimdi bunu görsel bir boyuta taşıyoruz. AykenOS kabuğu, gerektiğinde bir **grafiksel arayüz** oluşturup gösterebilecek yeteneğe kavuşacak. Konsept, kabukta girilen komutlarla bir **dashboard arayüzü tanımlamak** ve onun OpenGL gibi bir grafik API ile gerçek zamanlı çizimini sağlamaktır. Örneğin,
 - Kullanıcı `> ui.scene "sysdash"` yazarak yeni bir sahne başlatır (sistem durumu panosu oluşturmak istediğini belirtir).
 - Ardından `>> layout grid:2x2` komutuyla 2x2 bir panel düzeni belirler.
 - Devamında `>> widget "cpu" from:sys.hw.metrics['cpu']` ve benzeri komutlarla bu panoya CPU kullanım grafiği, disk I/O grafiği, ağ trafiği grafiği gibi widget'lar ekler.
 - Son olarak `>> render opengl:true` komutu verildiğinde, kabuk ortamı bu tanımlanan sahneyi çizmesi için grafik motorunu devreye sokar. Sonuç olarak ekranda gerçek zamanlı güncellenen bir sistem gösterge paneli çıkacaktır.

Bu sihir gibi görünen işlemin arka planında, kabuk yorumlayıcısı komutları bir **sahne grafiği (scene graph)** veri yapısında toplar. Ardından AykenOS içerisindeki bir **grafik çizim modülü** (muhtemelen kullanıcı modunda çalışan bir OpenGL/GLSL çizici) bu scene graph'ı alıp gerçek piksel çizimlerini yapar. AykenOS zaten temel bir framebuffer konsoluna sahip olduğundan, grafik mod için ya bir mode switch yapıp çift buffer'lı bir ekrana geçilebilir ya da mevcut konsolun bir bölümünde grafik

gösterilebilir. Teknik olarak en temiz yol, bir kullanıcı modu uygulamasının (örn. `ayken-ui`) framebuffer belleğine erişerek çizim yapması olacaktır. Bu uygulama OpenGL ES benzeri bir kütüphane kullanabilir veya doğrudan grafik framebuffer'ına yazabilir.

- **AI Destekli Görselleştirme:** Dashboard ve grafik arayüzü tanımlama işini de AI ile desteklemek mümkündür. Örneğin, kullanıcı `> pipeline[ ... >> ui.plot type:"graph" ]` gibi bir komutla veri işleme sonucu bir grafiğe gönderebilir. Bu durumda tinyLLM, gösterilecek veriye en uygun grafik türü veya shader konfigürasyonunu önermek için devreye girebilir. Hatta "Bu metrikleri en anlamlı nasıl göstereyim?" gibi bir soru sorulduğunda, AI uygun bir dashboard layout'u veya grafik tipi önerebilir. Bu özellikler fazlasıyla yenilikçi olduğundan, ilk iterasyonda küçük adımlarla yaklaşmak gerek. Örneğin, ilk olarak sabit tanımlı bir iki grafik widget'ı elle çizdirip göstermek başarıldıktan sonra, AI desteği basit bir senaryo için eklenebilir.
- **Kullanıcı Arayüzü ve Deneyim:** Bu faz AykenOS için ilk grafik arayüz denemesidir. Tam teşekküllü bir masaüstü ortamı değil, ancak komut satırı ile tanımlanan **canlı arayüzler** konsepti, geleneksel işletim sistemlerinde bile nadir görülen bir yaklaşımdır. Ekip için bu, **grafik programlama** (OpenGL/GLSL), **etkileşimli arayüz tasarımı** ve OS düzeyinde **gerçek zamanlı veri görselleştirme** konularında eğitim demektir. Projenin tanıtımı açısından da, AI destekli bir grafiksel dashboard gösterimi son derece etkileyici olacaktır (örneğin, AI'nın önerisiyle otomatik düzenlenen bir sistem izleme paneli).
- **Test ve Optimizasyon:** Bu özellikleri test etmek, belki de geliştirmek kadar zorlayıcı olabilir. Çünkü gerçek zamanda akan verilerin çizimi söz konusu, düşük performans kullanıcının hemen dikkatini çeker. İlk aşamada QEMU içinde yazılım tabanlı bir çizimle test yapılabilir. Çalışan bir CPU yük grafiği, bellek kullanım çubuğu vs. görmek mümkün olursa, gerçek donanımda (tercihen ekran çıkışı olan bir cihazda, örneğin HDMI bağlı bir RPi) denemeler yapılmalıdır. Ekip, OpenGL shader'larının ve çizim döngüsünün verimini artırmak için belki C ile optimize kütüphanelere ihtiyaç duyabilir. Ayrıca, bu grafik özelliklerini opsiyonel yapmak da mantıklı: Yani AykenOS'u ekran çıktı olmadan sadece seri konsoldan çalıştıran bir kullanıcı, bu özellik yok diye sistemin diğer kısımlarında problem yaşamamalı. Bu nedenle, kabuk **UI modunu algılayıp**, destek yoksa (veya istenmiyorsa) hiçbir OpenGL çağrısı yapmadan çalışmalarını sürdürmeli.

Özetle Faz 4, AykenOS'un **hem yatayda farklı donanımlara açıldığı**, hem de dikeyde kullanıcı etkileşimini zenginleştirdiği bir aşamadır. Başarılı olunduğunda, elimizde hem PC hem ARM gibi platformlarda çalışabilen, hem de komut satırından canlı grafiksel arayüz çıkışı alabilen bir sistem olacak. Bu, modüler tasarımın bir zaferidir: Kabuk dili aynı kalırken, çıktısı metin veya grafik olarak çeşitlendirilebilmiştir. AykenOS ekosisteminin bu esnekliği, ileride üçüncü parti uygulamaların da aynı veri/komut modelini kullanarak kendi arayüzlerini tanımlayabileceği anlamına gelir ki bu da vizyonumuzun bir parçasıdır (işletim sistemi üzerinde büyüyen modüler uygulamalar fikri).

Faz 5: Çoklu Platform Desteği ve Yaygınlaştırma

AykenOS'un çekirdek özellikleri ve yenilikçi yapısı büyük oranda şekillendiğinde, artık **projenin olgunlaştırılması ve farklı ortamlara uyarlanması** sürecine giriyoruz. Faz 5'in ana teması, **taşınabilirlik (portability)**, **optimizasyon** ve **yaygınlaştırma** olacaktır:

- **Mimari Desteğinin Tamamlanması:** Faz 4'te Raspberry Pi gibi ARM tabanlı bir cihazda temel testler yapılmışsa, şimdi bunu sistemli hale getirme zamanı. AykenOS'un kod tabanında mimariye özgü kısımlar net olarak ayrılmalı (muhtemelen `kernel/arch/x86_64`, `kernel/arch/arm64` gibi klasör ayrımları halihazırda var) ve her mimaride aynı işlevselliği sunacak şekilde eksik kısımlar tamamlanmalı. RISC-V desteği de denklemde unutulmamalı; özellikle gömülü sistemler ve yeni nesil cihazlar için RISC-V önem kazanıyor. Bootloader seviyesinde RISC-V desteğimiz var, ancak kernel tarafında belki RISC-V context switch, kesme mimarisi vs. ele alınmalıdır. Bu işler oldukça düşük seviye ve mimari başına uzmanlık gerektirdiğinden, ekip içi görev paylaşımı yapılarak farklı mimariler paralel olarak ele alınabilir.
- **Performans İyileştirmeleri:** Farklı platformlara geçerken, her biri için performans profilinin çıkartılması gerekir. Örneğin, `x86_64` üzerinde iyi çalışan bir bellek yöneticisi, ARM üzerinde farklı davranabilir (belki önbellek hizalamaları, sayfa boyutu farkları vs.). Aynı şekilde AI modülleri `x86_64`'te AVX destekli hızlı çalışırken, ARM'de NEON optimizasyonu gerektirebilir. Bu aşamada ekip, **profiling ve optimizasyon** konularına eğilmelidir. Rust ile yazılmış kritik bölümler varsa bunların `unsafe` optimizasyonları gözden geçirilebilir, C tarafında derleyici optimizasyon seviyeleri artırılabilir. Hatta ileride AI modülleri için cihazların GPU kapasitelerinden faydalanmak (CUDA, OpenCL, Vulkan Compute gibi) gündeme gelebilir – gerçi bu belki Faz 6'nın konusu olacak.
- **Modülerlik ve Paketleme:** AykenOS büyüdükçe, sistemin modüler yapısını korumak önemli. Bu nedenle, çekirdek, kabuk, AI core, vs. her biri ayrı derlenip bir araya gelen parçalar şeklinde kalmalı. Belki bu aşamada bir **paket yöneticisi veya modül yükleme mekanizması** tasarlamak değerlendirilebilir. Örneğin, AI modelleri veya veri modülleri OTA (over-the-air) güncellemelerle değiştirilebilsin istenir. Bu yüzden, AykenOS'un modüllerini tanıyan basit bir paket sistemi veya en azından versiyonlama stratejisi geliştirilmelidir. Teknik ekip, versiyon kontrol, sürekli entegrasyon (CI) ve otomatik derleme testlerini de bu evrede iyice oturtmalıdır.
- **Dokümantasyon ve Eğitim:** Proje bu noktaya geldiğinde, dış katkılara açılmaya hazır hale gelebilir. AykenOS ikili lisans modeliyle dağıtılacak olsa da (ASAL topluluk lisansı ve ticari ACL lisansı), daha fazla geliştiricinin katılımı veya geri bildirim alabilmek için açık dokümantasyon kritik önem taşır. Teknik ekip, **geliştirici dokümantasyonu** hazırlamalıdır:
 - API dokümanları (özellikle AI ve veri katmanı API'leri),
 - Örnek uygulama yazma rehberleri,
 - Sistem mimarisi açıklamaları (belki bu raporun bir özeti niteliğinde teknik makaleler) vs.

Aynı zamanda son kullanıcıya yönelik bir kılavuz da (ör. "AykenOS Shell Kullanım Kılavuzu") düşünülmelidir. Bu tür dokümantasyon, projenin tanıtımında bize rehber olacak ve konseptin öğrenilmesini kolaylaştıracaktır, zira AykenOS alışılmışın dışında kavramlar içeriyor.

- **Topluluk ve Ekosistem Oluşturma:** AykenOS'un değer önerilerinden biri, **standart işletim sistemi üzerinde büyüyen modüler uygulamalar** için bir zemin sunmasıdır (kullanıcının anlatımıyla, "işletim sistemi kademesinde rust+c ile performanslı, kolay çalıştırılabilir bir yapı oluşturma" ve uygulamaları buna bağımlı kılma hedefi). Bunu gerçekleştirebilmek için, AykenOS üzerinde çalışacak bazı **örnek üçüncü parti uygulamalar veya hizmetler** geliştirmek gerekebilir. Bu aşamada, belki küçük çaplı pilot projeler yapılabilir:

- Örneğin, bir **not alma uygulaması**, ancak klasik metin dosyası yerine AykenOS veri konteynerlerini kullanıyor ve AI ile not özetleme özelliğine sahip.
- Veya basit bir **web sunucusu**, AykenOS'un AI özelliklerini kullanarak akıllı log analizi yapan.
- Ya da bir **grafik/demo uygulaması**, OpenGL dashboard özelliğini kullanarak görsel bir şov yapıyor.

Bu örnekler, hem sistemin gücünü gösterir hem de eksik noktaları ortaya çıkarır. Ekip, bu pilot uygulamaları yaparken AykenOS'un API'lerini test etmiş ve geliştirmiş olacak.

Faz 5'in sonunda, AykenOS'un **birden fazla mimaride çalışan**, temel sistem servisleri optimize edilmiş, dokümantasyonu ve kullanım örnekleri ile desteklenmiş bir **beta sürümü** ortaya çıkmış olmalıdır. Bu, projenin dış dünyaya açılma ve geri bildirim alma evresidir. Teknik ekip için de farklı alanlarda uzmanlaşma fırsatı sunar: düşük seviye gömülü programlama, yüksek seviye AI entegrasyonu, sistem optimizasyonu, belge yazımı, topluluk yönetimi gibi.

Faz 6: İleri Düzey Uygulama Desteği ve Gelecek Vizyonu

AykenOS'un yol haritasındaki son faz, projeyi olgunlaştırıp gerçek dünya kullanımına yaklaştırmayı amaçlar. Bu aşamada yapılacak işler, önceki fazlardaki her bileşeni daha ileri seviyeye taşımak ve **vizyonu tam anlamıyla hayata geçirmek** için gerekli kalan parçaları tamamlamaktır:

- **Ağ Desteğinin Eklenmesi:** Modern bir işletim sistemi için ağ iletişimi şarttır. AykenOS'ta da hem geleneksel amaçlar (ör. internet bağlantısı, SSH erişimi) hem de AI ajanlarının harici veri çekebilmesi için bir **TCP/IP ağ yığını** entegre edilmelidir. Bu, belki sıfırdan değil ama mevcut hafif bir TCP/IP kütüphanesini porta ederek yapılabilir. Örneğin LWIP gibi bir kütüphane çekirdeğe entegre edilebilir veya kullanıcı modunda bir servis olarak çalıştırılabilir. Ağ desteği geldiğinde, AykenOS'un yapay zekâ yetenekleri bulutla da entegre edilebilir hale gelir (tabii lisans kısıtları dahilinde). Örneğin, güncelleme kontrolü, harici AI modellerinin indirilebilmesi, farklı cihazların birbirine AykenOS üzerinden bağlanması gibi senaryolar mümkün olur.
- **Yüksek Seviyeli Dil/Ortam Desteği:** AykenOS baştan aşağı Rust/C/C++ ile yazılıyor olsa da, kullanıcı alanında **farklı dillerin çalışabilmesi** önem kazanabilir. Özellikle proje, Blender gibi karmaşık uygulamaların çalışma biçiminden bahsetti **【user】** . Blender örneğinde, uygulamanın kendi içinde Python yorumlayıcı ve çeşitli kütüphane bağımlılıkları var. AykenOS yaklaşımı ise bazı bağımlılıkları OS seviyesinde sunarak bu yükü azaltmak olabilir. Örneğin:
 - AykenOS kabuğu ve veri sistemi halihazırda güçlü bir betikleme ve sorgulama imkânı sunuyor. Belki Blender benzeri bir uygulama, Python'a ihtiyaç duymadan AykenOS'un API'lerini kullanarak eşdeğer script fonksiyonlarını gerçekleştirebilir.
 - Yine de, yaygın dillerin desteği ekosistem için önemlidir. Bu yüzden ileride **Python yorumlayıcısı** veya **WebAssembly runtime** gibi bileşenler AykenOS'a entegre edilebilir. Hatta AI modülü sayesinde, kaynak koddan makine koduna çeviri veya anında yorumlama konularında yenilikçi çözümler

üretilebilir (örneğin AI'ın basit bir Python kodunu içsel olarak anlaması ve çalıştırması gibi deneysel özellikler).

- Bu aşamada amaç, geliştiricilerin AykenOS üzerinde uygulama geliştirirken **alışık oldukları araçları** mümkün olduğunca kullanabilmelerini sağlamak, ancak aynı zamanda onları AykenOS'un yeni paradigmalarıyla tanıştırmaktır. Örneğin, klasik bir POSIX C programı AykenOS'ta derlenip çalışabilir olmalı (temel libc ve syscalls desteği tamamlanarak), fakat geliştirici dilerse AykenOS'un veri-odaklı API'lerini de kullanabilmeli.
- **Arayüz ve Kullanıcı Deneyiminin İyileştirilmesi:** Faz 4'te başlatılan grafiksel arayüz kabiliyetleri genişletilebilir. Dashboard konsepti dışında, belki tam teşekküllü bir **pencere yöneticisi** değil ama veri odaklı etkileşimleri kolaylaştıran görsel araçlar eklenebilir. Örneğin:
 - Belirli veri tipleri için özel görüntüleyiciler (tablo tipi için tablo görünümü, graf tipi için diyagram çizici, text için zengin metin editörü gibi).
 - AI ile kullanıcı etkileşimini kolaylaştıran arayüz öğeleri (mesela, kabuktan bir komut yerine bir **soru-cevap sohbet penceresi** aracılığıyla sistemi yönetme imkânı).
 - Mobil cihaz hedefleniyorsa dokunmatik uyumlu arayüz elemanları.

Bu geliştirmeler sırasında hep hibrit yaklaşım korunmalıdır: AykenOS'un kabuk dili ve AI yetenekleri arka planda çalışmaya devam ederken, bunların üstüne gerekirse farklı arayüzler giydirilebilir. Basit bir örnek, bir web arayüzü ile AykenOS kabuğunu uzaktan yönetmek olabilir. HTTP üzerinden gelen istekleri kabuk komutu olarak yorumlayıp sonuçları dönen bir aracı (gateway) yapılabilir, böylece herhangi bir cihazdan tarayıcı ile AykenOS'un AI destekli gücüne erişilebilir.

- **Güvenlik ve Kararlılık:** Sistem büyüdükçe ve özellikle ağ desteğiyle dışa açıldığında, **güvenlik** en önemli konulardan biri olacak. AykenOS'un baştan AI odaklı oluşu, bazı güvenlik risklerini de beraberinde getiriyor (LLM'lerin beklenmedik çıktılar üretmesi, veri sızıntıları vs.). Bu nedenle:
 - Bir **politika motoru** geliştirilmeli ve AI'nın yapabileceklerine dair kurallar tanımlanmalı. Örneğin, AI önerileri belirli komutları içeriyorsa (dosya silme, sistem ayar değiştirme gibi) bunları ancak belirli koşullarda (kullanıcı interaktif onayı, veya admin rolü) uygulamaya sokan bir katman olmalı.
 - Kullanıcı yönetimi ve yetkilendirme sistemi eklenmeli. Çok kullanıcı bir senaryo belki uzak, fakat en azından "admin" vs "standart" kullanıcı ayrışa bile kabuk ve AI davranışları buna göre ayarlanabilir (docx içindeki JSON örneğinde `can_modify` gibi bir alan görüyoruz, bu tarz poliçeler gerçek anlam kazanacak).
 - Kararlılık için ise yoğun test, bellek sızıntısı analizleri, stress-test yapılması şart. AI ve grafik özellikleri bellek tüketimini artıracığından, hafıza yönetimi ve olası bellek sızıntıları tespit edilip giderilmeli. Rust kullanımı burada avantaj, ancak C kısımlar için de statik analiz araçları (AddressSanitizer, Valgrind) devreye sokulabilir.
- **Vizyonun Tamamlanması:** Bu aşamada AykenOS'un ilk konseptte belirlenen tüm parçaları birleşmiş olacak. Tek cümleyle ifade etmek gerekirse: AykenOS, *yapay zekânın birinci sınıf vatandaşı olduğu*, veriyi merkeze alan ve kendi kendini geliştirme potansiyeli taşıyan bir mikro işletim ortamı olacaktır. Artık klasik bir kabuk veya OS olmaktan çıkıp kullanıcının **düşünce ortağı, yaratıcı hızlandırıcısı** haline gelme hedefine yaklaşmıştır [medium.com](https://medium.com/medium.com).

Bu noktada geriye kalan, vizyonu gösterecek örnek senaryoların, tanıtım materyallerinin hazırlanması ve belki de **sürüm 1.0** için son rötuşların yapılmasıdır. Örneğin,

- AykenOS'un kendini tanıttığı etkileşimli bir başlangıç senaryosu yazılabilir (kullanıcı ilk oturum açtığında sistemin AI ajanı onu yönlendirir, neler yapabileceğini anlatır).
- Proje için bir **spec v1.0** dokümanı yayınlanır (docx'te bahsedilen v0.1 taslağının geniş hali), böylece hem iç ekip hem dış geliştiriciler için sistemin resmi tanımı olur.
- Son olarak lisanslama ve dağıtım stratejileri uygulanır (topluluk sürümü vs. ticari sürüm ayarlamaları). Gerekirse, ticari kullanılacak kısımlarda daha gelişmiş model entegrasyonları veya özel araçlar devreye girebilir.
- **Özet ve Öngörüler:** AykenOS projesi başlangıçta avant-garde bir fikir olarak doğdu ve bu yol haritası boyunca adım adım gerçeğe dönüştü. Her fazda modüler bir yaklaşım benimsendiği için, hibrit yapı hiç kaosa mahal vermeden büyüdü. Erken aşamalarda sınırlı tutulan veri türleri ve tek bir LLM gibi kararlar, sistemin kademeli evrimini mümkün kıldı. Sonuçta ortaya çıkan şey sadece bir shell ya da bir OS değil, belki de “**AI-native micro operating environment**” olarak adlandırılabilir bambaşka bir kategoridir. Bu sistemde dosyalar yerine anlamlı veri nesneleri, komutlar yerine amaçlar ve sorular, kullanıcı yerine ortak çalışan AI ajanları vardır. AykenOS ekibi için bu yolculuk, işletim sistemlerini yeniden düşünme ve geleceğin bilgisayar etkileşimini şekillendirme fırsatıdır. Yol haritasına sadık kalarak ve her adımda gerekli bilgi birikimini derinleştirerek ilerlersek, AykenOS vizyonunu başarıyla hayata geçireceğimiz konusunda güçlü bir inancımız olabilir.

Kaynaklar:

- AykenOS Proje Dökümanları:
 - README (Proje Özeti ve Özellikler)
 - Proje Dizin Yapısı (Teknik Yapı)
 - Proje Durum Raporu (Mevcut Durum ve Eksikler)
 - İnovatif Kabuk ve Veri Sistemi Taslak Dokümanı
- İlgili Konsept Yazıları:
 - *The AI-Native OS: Rethinking the Operating System* – AI odaklı işletim sisteminin temel ilkeleri medium.com/medium.com (AykenOS vizyonuyla paralellik göstermektedir).